Rapport de test statique - Stayora

Analyse exécutée le 11 octobre 2026. Résultat : **53 fichiers analysés, 0 erreur et 1 avertissement**. Aucun correctif n'a été appliqué pendant l'analyse.

1. Objectif et présentation du projet

Ce travail académique présente un outil de test statique, l'exécute sur un périmètre significatif du projet local et conserve les preuves permettant de vérifier ses résultats.

Stayora est une application de location d'hébergements : interface Angular, serveur Spring Boot et base PostgreSQL Neon. Les cinq modules métier sont les propriétés, les réservations, la disponibilité, les avis et la messagerie. Les comptes constituent une infrastructure commune. Le dossier examiné est `C:\Users\everp\Documents\GitHub\web-ai`.

Le projet combine notamment TypeScript, HTML, CSS, Java et SQL. L'outil choisi couvre ici le frontal Angular. Il ne constitue donc pas une analyse statique exhaustive de toute l'application.

2. Définition du test statique

Un test statique examine le code et ses structures sans exécuter les fonctionnalités de l'application. ESLint analyse la syntaxe et applique des règles configurées pour repérer certaines erreurs, pratiques risquées ou incohérences de maintenance.

Un test dynamique exécute, au contraire, du code : test unitaire, appel d'API, scénario de réservation ou interaction dans un navigateur. Les vérifications fonctionnelles réalisées pendant l'implémentation sont distinctes du présent travail. Aucun serveur, aucune application et aucun test dynamique n'ont été lancés pendant cette phase d'analyse.

3. Outil retenu et justification

ESLint 8.57.1, déjà installé et configuré dans le projet, est adapté au TypeScript et aux modèles Angular grâce aux extensions présentes. Il permet une analyse locale, rapide, sans compte, clé, serveur d'analyse ni téléversement du code. Il est plus simple ici de réutiliser cet environnement que d'installer une autre infrastructure.

Versions effectivement utilisées ou observées dans les dépendances verrouillées :

Élément	Version
Node.js exécutant l'analyse	22.23.3
ESLint exécuté	8.57.1
Analyseur et extension TypeScript ESLint	8.18.0
Extensions Angular ESLint	18.4.3
Angular du projet	18.2.14
TypeScript du projet	5.5.4

Le serveur utilise Spring Boot 3.3.6 et Java 17, mais ces fichiers ne sont pas analysés par ESLint. La version ESLint 8 est ancienne et sa maintenance générale a pris fin le 5 octobre 2024. Elle est conservée pour respecter l'environnement existant ; aucune mise à niveau n'a été réalisée pour ce travail. Documentation officielle de la ligne de commande ESLint 8.

4. Préservation de l'état examiné

La demande de développement du projet et la consigne de préserver les fichiers pendant le test statique ont été traitées en deux phases. Les fonctionnalités ont d'abord été implémentées et vérifiées. Puis leur état a été figé avant l'analyse. Les fichiers déjà modifiés ou ajoutés à ce moment n'ont pas été réinitialisés.

Le commit de référence est `0f8496538bde0a890cbbf2f4cb8f3b55e1792b6a`, mais l'analyse porte sur l'arbre de travail final, comprenant ses modifications non commitées, et non sur ce commit seul. Les états Git exacts sont conservés dans `git-avant.txt` et `git-apres.txt`.

Les empreintes SHA-256 de **196 fichiers suivis ou nouveaux non ignorés**, hors dossier de ces livrables, sont identiques avant et après l'exécution. Les manifestes `integrite-avant.json` et `integrite-apres.json` couvrent notamment sources, configurations, fichiers de dépendances et documents préexistants. Les dépendances installées, fichiers générés et secrets ignorés ne sont pas inclus dans cette vérification ; elle ne prétend pas établir leur immutabilité.

Pendant cette phase : aucun `--fix`, aucun formatage en écriture des fichiers du projet, aucune modification des dépendances, aucun commit, aucune publication et aucun envoi du code vers un service d'analyse. Seul le nouveau dossier de livrables a été enrichi. La consultation de documentation publique n'a pas transmis les sources du projet.

5. Installation et configuration réellement utilisées

Aucune installation n'a été nécessaire pendant l'analyse. Le binaire Node 22 existant et le fichier `frontend/node_modules/eslint/bin/eslint.js` ont été exécutés directement. Le collecteur `analyse.mjs` ne lance pas les scripts applicatifs : il lit Git et les fichiers, appelle ESLint puis archive ses résultats.

La configuration séparée `eslint-rapport.cjs` hérite de `frontend/.eslintrc.json`. Elle conserve les règles Angular, le contrôle des composants autonomes, l'interdiction des types `any` explicites et les règles HTML/accessibilité existantes. Le processeur Angular configuré extrait les modèles HTML intégrés dans les composants TypeScript.

Des contrôles de maintenance sont ajoutés sans désactiver les règles existantes : variables inutilisées adaptées à TypeScript, égalité stricte et console au niveau avertissement ; débogueur, évaluation dynamique, construction dynamique de fonctions, code inaccessible et conditions constantes au niveau erreur. Les configurations effectives TypeScript et HTML sont archivées dans les fichiers `configuration-*-stdout.json`.

Cette configuration de rapport est plus exigeante que la configuration applicative habituelle. Son unique avertissement ne contredit donc pas le résultat du lint habituel. Il provient d'une règle ajoutée pour cet examen, et non d'un échec d'installation ou d'une règle désactivée pour masquer des défauts.

6. Périmètre et exclusions

La cible est l'ensemble de `frontend/src` pour les extensions `.ts` et `.html` : **52 fichiers TypeScript et 1 fichier HTML**, soit **5 732 lignes physiques selon le comptage du collecteur**. Ce comptage inclut les lignes vides et la ligne vide finale lorsqu'elle existe ; il ne mesure pas uniquement des lignes de logique métier.

Les 53 fichiers ciblés correspondent exactement aux 53 chemins retournés par ESLint. Quinze fichiers sont des spécifications `.spec.ts`, examinées comme du texte source sans exécution. Les modèles HTML intégrés sont traités avec leur composant et ne sont pas comptés comme des fichiers physiques supplémentaires.

Partie du frontal	Fichiers ciblés
Propriétés et recherche - Aymen	12
Réservations et paiements - Hadir	9
Disponibilité et tableau de bord - Rania	4

Partie du frontal	Fichiers ciblés
Avis et étoiles - Wassim	6
Conversations et messages - Kawther	4
Comptes partagés	3
Contrats et services communs	4
Composants communs	7
Entrées et fichiers à la racine du frontal	4
Total	**53**

Les répertoires `node_modules`, `dist`, `.angular`, `target`, les fichiers de couverture et les journaux temporaires sont hors cible. Les références de conception, documents, ressources publiques et secrets ne sont pas analysés. Le Java du serveur, les migrations SQL, le CSS/Tailwind, les scripts racine et la configuration d'intégration continue sont également hors périmètre. Aucun résultat de sécurité ou de qualité global ne peut être déduit pour ces éléments.

7. Commande, chronométrage et preuves

Dossier d'exécution : `C:\Users\everp\Documents\GitHub\web-ai\frontend`.

Commande exacte de l'analyse, reconstruite à partir de l'exécutable et des arguments réellement transmis au processus :

```
& 'C:\custom\bins\node\node-v22.23.3-win-x64\node.exe'
'C:\Users\everp\Documents\GitHub\web-ai\frontend\node_modules\eslint\bin\eslint.js' '--config'
'C:\Users\everp\Documents\GitHub\web-ai\docs\static-testing-2026-10-11\eslint-rapport.cjs'
'--resolve-plugins-relative-to' 'C:\Users\everp\Documents\GitHub\web-ai\frontend' '--ext'
'.ts,.html' '--format' 'json' '--report-unused-disable-directives' 'src'
```

Mesure de l'analyse principale	Valeur réelle
Début UTC	2026-10-11T19:17:21.867Z
Fin UTC	2026-10-11T19:17:24.832Z
Début dans le fuseau de consignation	11/10/2026, 20:17:21 UTC+01:00
Fuseau de consignation explicite	Etc/GMT-1, décalage fixe UTC+01:00
Durée du processus mesurée	2,963 secondes
Code de sortie réel d'ESLint	0
Erreur de lancement	Aucune
Taille de la sortie d'erreur	0 octet

`execution.json` conserve aussi les commandes exactes, horaires, durées et codes de sortie de la vérification des versions et des deux consultations de configuration. La durée mesurée utilise une horloge monotone ; les dates servent à situer l'exécution.

`analyse-stdout.json` contient la sortie standard JSON brute ; `analyse-stderr.txt` contient la sortie d'erreur brute, vide. Le collecteur enregistre directement le code du processus ESLint, sans le remplacer par celui d'une redirection ou d'un pipeline.

8. Résultats réels et constat expliqué

Indicateur	Résultat
Fichiers effectivement analysés	53
Fichiers contenant un constat	1
Erreurs	0
Erreurs fatales d'analyse	0
Avertissements	1
Constats de la règle `no-console`	1

Usage de la console dans le démarrage Angular

- **Fichier :** `frontend/src/main.ts`.
- **Position réelle :** ligne 20, colonne 16 ; fin à la colonne 29.
- **Règle :** `no-console`.
- **Gravité :** avertissement, niveau 1.
- **Message, traduit :** « Instruction console inattendue. » Le message original, inchangé, figure dans les journaux JSON.
- **Instruction concernée :** le traitement d'échec du démarrage appelle `console.error` avec un message fixe.

La règle signale l'utilisation de la console dans le code du navigateur. Une journalisation centralisée ou un état visible de démarrage pourrait rendre le comportement plus cohérent et plus facile à contrôler. Règle officielle ESLint `no-console`.

Interprétation prudente : cet appel est un diagnostic volontaire d'échec d'initialisation, et son message fixe ne contient pas de donnée personnelle ni de secret. Le constat n'établit ni une panne ni une vulnérabilité. Il peut représenter un compromis de conception acceptable plutôt qu'un défaut à supprimer. La décision doit tenir compte de la visibilité nécessaire des échecs de démarrage.

Le code de sortie 0 signifie que l'analyse ne contient pas d'erreur bloquante selon la configuration utilisée. Les avertissements sont autorisés par défaut ; aucune option imposant zéro avertissement n'a été transmise. Ce code ne signifie donc pas « aucun constat ».

9. Améliorations recommandées, non appliquées

1. Examiner le diagnostic de démarrage avec l'équipe : conserver une information utile, éventuellement au moyen d'un état visible et d'une journalisation contrôlée, sans supprimer aveuglément la gestion d'erreur. Cette recommandation n'a pas été appliquée.

2. Discuter de l'intégration des règles de maintenance du rapport dans la configuration habituelle et l'intégration continue. Ne pas neutraliser une règle uniquement pour obtenir un compteur nul.

3. Prévoir ultérieurement une analyse statique spécifique du Java et des migrations SQL : le présent outil ne les couvre pas.

4. Étudier une évolution coordonnée des versions et des outils avant une exploitation publique, en respectant les contraintes de compatibilité. Aucune dépendance ni version n'a été changée pendant cet examen.

10. Limites et conclusion

L'analyse n'utilise pas les informations de typage avancées d'un projet TypeScript complet ; elle n'est pas équivalente à une compilation ni à une preuve formelle. Ses règles d'accessibilité ne remplacent pas une évaluation complète au clavier ou avec un lecteur d'écran. Elle ne vérifie ni les transactions PostgreSQL, ni les permissions à l'exécution, ni les totaux de réservation, ni les réponses d'un fournisseur d'IA, ni le rendu visuel.

L'absence d'erreurs signalées ne démontre pas l'absence de bogues ou la sécurité globale du logiciel. L'unique avertissement mérite une décision de maintenance argumentée. Les **53 fichiers analysés**, le **code de sortie 0**, la **durée de 2,963 secondes** et les **196 empreintes inchangées** sont vérifiables dans les livrables. Aucune correction n'a été réalisée pendant l'analyse.

11. Documentation officielle consultée

- Ligne de commande ESLint 8 : options, format JSON, consultation de configuration, codes de sortie et statut de maintenance.
- Règle `no-console` : fonctionnement et interprétation du constat.
- Règle TypeScript `no-unused-vars` : contrôle adapté aux constructions TypeScript.
- Dépôt officiel Angular ESLint : analyse des composants et modèles Angular.
- Compatibilité officielle Angular : contexte de compatibilité de l'environnement du projet.

Ces références ont été consultées le 11 octobre 2026. Les versions, fichiers, messages et nombres de ce rapport proviennent des dépendances et de l'exécution locales, pas d'exemples issus de ces pages.

12. Contenu des livrables

- `rapport.md` et `rapport.pdf` : rapport en français et version imprimable.
- `reproduire.md` : guide de reproduction et lecture des résultats.
- `analyse.mjs` et `eslint-rapport.cjs` : collecteur et configuration séparée.
- `execution.json` : commandes, versions, horaires, durées, codes et compteurs réels.
- `analyse-stdout.json`, `analyse-stderr.txt` et `version-*-stdout.txt`/`version-*-stderr.txt` : sorties brutes.
- `configuration-*-stdout.json`/`configuration-*-stderr.txt` : configurations effectives et leurs journaux.
- `fichiers-cibles.json`, `integrite-avant.json`, `integrite-apres.json`, `git-avant.txt`, `git-apres.txt` : périmètre et préservation de l'état examiné.

Les journaux bruts conservent les messages de l'outil dans leur langue originale afin de ne pas altérer les preuves. Leur interprétation et le présent rapport sont en français.